KOCAELİ ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

BİLİŞİM SİSTEMLERİ MÜHENDİSLİĞİ

Yazılım Geliştirme Laboratuvarı-II Dersi

Proje - 1

Proje; modern yazılım geliştirme süreçlerinin temelini oluşturan Mikroservis Mimarisi ve servisler arası trafik yönetimini sağlayan bir Dispatcher (API Gateway) yazılımının uçtan uca geliştirilmesini amaçlamaktadır. Öğrenciler, sistem bileşenlerini bağımsız servisler olarak kurgulayacak, tüm dış istekleri merkezi bir Dispatcher üzerinden yöneten, güvenli ve ölçeklenebilir bir uygulama tasarlayacaklardır. Bu süreçte Dispatcher biriminin, hata payını minimize eden Test-Driven Development (TDD) disipliniyle geliştirme yapacaklardır. Proje; nesne yönelimli programlama, ağ protokolleri, servis orkestrasyonu ve modern test süreçlerini bütünleştirici biçimde ölçmeyi hedeflemektedir.

1. Proje Konusu

Proje; modern yazılım geliştirme süreçlerinin temelini oluşturan **Mikroservis Mimarisi** ve servisler arası trafik yönetimini sağlayan bir **Dispatcher (API Gateway)** yazılımının uçtan uca geliştirilmesini amaçlamaktadır. Öğrenciler, sistem bileşenlerini bağımsız servisler olarak kurgulayacak, tüm dış istekleri merkezi bir Dispatcher üzerinden yöneten, güvenli ve ölçeklenebilir bir uygulama tasarlayacaklardır.

Bu süreçte Dispatcher biriminin, hata payını minimize eden ve kod kalitesini artıran **Test-Driven Development (TDD)** disipliniyle geliştirilmesi zorunludur. Proje; **nesne yönelimli programlama**, **ağ yönetimi**, **servis orkestrasyonu**, **veri yapıları** ve **modern yazılım tasarımı** becerilerini bütünleştirici biçimde ölçmeyi hedeflemektedir.

2. Genel Gereksinimler

1. Proje **2 kişilik gruplar** halinde yapılacaktır. Grup oluşturma linki için tıklayınız.
2. Proje kodları, çalışan uygulama ve GitHub üzerinde Markdown ve Mermaid kullanılarak yazılmış olan proje raporu tek bir sıkıştırılmış dosya halinde teslim edilecektir. Rapor içerisinde sınıf diyagramları, iş akışları, algoritmaların akış diyagramları, varsa veri tabanı E-R diyagramı vb. tüm görseller açıkça gösterilmelidir.
3. Rapor, sistemin tasarımını, kullanılan algoritmaları, testleri ve ekran görüntülerini içermelidir.
4. Son grup oluşturma tarihine kadar grup oluşturma işlemini tamamlamayan öğrencilerin projeleri değerlendirmeye alınmayacaktır. Sürelerin kaçırılması halinde öğrencinin projeye devam etmediği değerlendirilecek ve bu kural esnetilmeyecektir. Tek başlarına olsalar bile ilgili tarihe kadar dokümana bilgilerini işlememiş öğrencilerin projeleri **0 (sıfır)** olarak değerlendirilecektir.
5. Dispatcher servisinin geliştirilmesinde TDD (Red-Green-Refactor) yaklaşımı zorunludur. Dispatcher için yazılan test dosyalarının zaman damgası, fonksiyonel kodlardan önce gelmelidir. Aksi halde proje **0 (sıfır)** olarak değerlendirilecektir.
6. GitHub üzerinde her ekip üyesinin eşit ağırlıkla ve düzenli olarak commit işlemi gerçekleştirmesi gerekmektedir. Aksi halde proje **0 (sıfır)** olarak değerlendirilecektir.
7. Uygulamanın tamamı nesne yönelimli programlama (OOP) prensiplerine (SOLID, soyutlama, arayüz kullanımı vb.) uygun geliştirilmelidir. OOP standartlarına uymayan projeler kabul edilmeyecektir. Aksi halde proje **0 (sıfır)** olarak değerlendirilecektir.
8. Yük testlerinin JMeter, Locust veya k6 gibi profesyonel araçlarla yapılmaması ve rapora bu araçların kullanım senaryoları ile sonuçların eklenmemesi halinde projeler kabul edilmeyecektir. Aksi halde proje **0 (sıfır)** olarak değerlendirilecektir.

3. İşlevsel İsterler

Sistem, bir Dispatcher, bir oturum açma ve en az iki mikroservis olmak üzere en az 4 tane bağımsız üniteden oluşmalıdır.

3.1. Dispatcher (API Gateway) ve TDD

Dispatcher, sistemin tek giriş noktasıdır. Bu servisin geliştirilmesinde TDD (Red-Green-Refactor) döngüsü uygulanmalıdır.

İstekleri URL yapısına göre ilgili mikroservise yönlendirmelidir.

Yetkilendirme kontrolleri tek merkezden yapılmalıdır. Yetkiler ve erişim bilgileri bir NoSQL veri tabanı üzerinden tutulmalıdır.

Tüm trafik ve yönetici işlemleri loglanmalı, ulaşılamayan servisler ve hatalar için uygun hata kodları (HTTP 4xx, HTTP 5xx vb.) dönülmelidir. Örneğin hata oluştuğunda bile her zaman HTTP 200 OK dönüp, JSON gövdesinde {"error": true}şeklinde mesaj dönülmemelidir.

3.2. Mikroservisler

Yoğun trafiğe maruz kalabilecek bir senaryo tasarlanmalı ve bu senaryoya ait en az 2 adet işlevsel mikroservis geliştirilmelidir (Örn: Message Service, User Service, Product Service, Report Service vb.).

Her servisin kendine ait bağımsız bir NoSQL veri saklama yapısı bulunmalıdır.

Yine servisler arası veri transferi JSON formatında sağlanmalıdır.

Tüm yetkilendirme ve yönlendirme mantığı mikroservislerin içine gömülerek, Dispatcher sadece bir "proxy" olarak bırakılmamalıdır. Yetkilendirme (Auth) mantığının sadece Dispatcher'da olması ve mikroservislerin Dispatcher dışından gelen istekleri reddetmesi (Network Isolation) gerekmektedir. Yani mikroservislerin sadece iç ağda olması, dış dünyaya yalnızca Dispatcher'ın açık olması sağlanmalıdır. Bu durumun nasıl sağlandığı ekran görüntüleri de kullanılarak raporda belirtilmelidir.

4. Teknik İsterler

4.1. Sistem Tasarımı

- Nesne yönelimli geliştirme zorunludur. Prensiplere uymayan kodlar **0 (sıfır)** olarak değerlendirilecektir.
- Yapı Dockerize edilmelidir. Tüm mimari *docker-compose up* komutuyla tek seferde ayağa kalkmalıdır.
- Programlama dili kısıtı yoktur; ancak seçilen dilin TDD uyumlu bir test framework'üne (örn: Pytest, JUnit, xUnit, Jest) sahip olması şarttır.
- Geliştirilecek mikroservislerin API tasarımlarında Richardson Olgunluk Modeli (Richardson Maturity Model – RMM) Seviye 2 standartlarına uyulması beklenmektedir. Kaynakların (Resource) URI üzerinden tanımlanması ve işlemlerin (Create, Read, Update, Delete) uygun HTTP metotları (POST, GET, PUT, DELETE) ve doğru durum (HTTP 4xx, HTTP 5xx gibi) kodları ile gerçekleştirilmesi zorunludur. Özet olarak, tüm istekler örneğin POST ile gönderilip, URL içinde "…/deleteUser?id=1" gibi parametreler kullanılmamalıdır.

4.2. Veri İşlemleri

- Tasarladığınız senaryoya ve servislerin yapısına uygun bir NoSQL veri tabanı sistemi kullanılmadır.
- Veri tabanı işlemleri için kullanılacak NoSQL sisteminin bilinen, dokümante edilmiş bir veri tabanı motoru olması gerekmektedir. Gerçek bir NoSQL veri tabanı (Redis, Mongo, Cassandra vb.) yerine, bir JSON dosyasını okuyup yazan basit bir sınıf geliştirilmemelidir.
- Dispatcher da bir "ünite/servis" olarak tanımlandığından, onun veri tabanının da diğer mikroservislerden tamamen izole olması gerekmektedir.
- Servisler arasındaki veri aktarımları JSON ile gerçekleştirilmelidir.

4.3. Kullanıcı Arayüzü

- Programlama dili kısıtı bulunmamaktadır. Ancak dispatcher üzerindeki trafik akışı bir grafiksel arayüz (Grafana vb.) ve detaylı log tablosu ile sunulmalıdır.

4.4. Performans & Yük Testi

- Sistem, JMeter, Locust veya k6 gibi profesyonel araçlar kullanılarak simüle edilen yoğun istek trafiği altında test edilmelidir.
- Dispatcher'ın bu eş zamanlı istekleri karşılama başarısı (yönlendirme doğruluğu ve yanıt süreleri vb.) ölçülmelidir.
- Yapılan testlerin sonuçları (örneğin: 50, 100, 200, 500 eş zamanlı istekte ortalama yanıt süreleri, hata oranları vb.) kullanıcı ara yüzünde ve raporda sunulmalıdır.

5. Rapor İsterleri

Rapor aşağıdaki bölümleri içermelidir. GitHub üzerinde readme.md dosyasında Markdown kullanılarak yazılmalıdır.

1. Proje adı, ekip üyeleri, tarih.
2. Problemin tanımı, amaç vb. bilgilerin bulunduğu giriş kısmı.
3. Projede gerçeklenmesi istenilen tasarımın ve mikroservislerin, Richardson Olgunluk Modelini, Restful servislerin ne olduğunu, bu servislerdeki sınıf yapıları ve sequence diyagramları ve varsa algoritmaları anlatan bir bölüm. Her öğenin çalışma mantığı, akış diyagramı, sequence diyagramı vb. (Mermaid kullanılarak), karmaşıklık analizi ve literatür incelemesi.
4. Projenin yapılarının ve modüllerin Mermaid ile gösterilmesi ve işlevlerin açıklanması.
5. Uygulamaya ait açıklamalar ekran görüntüleri, test senaryoları ve sonuçlar.
6. Başarılar, sınırlılıklar, olası geliştirmelerin belirtildiği sonuç ve tartışma bölümü.

6. Değerlendirme Kriterleri (100 Puan)

- Dispatcher İşlevselliği
- TDD’nin uygulanması
- Mikroservis Mimarisi ve Veri İzolasyonu
- RMM’nin uygulanması (Seviye 3 uygulanırsa +5 puan)
- Docker ve Sistem Orkestrasyonu
- Görselleştirme
- OOP Tasarım ve Mimari
- Veri Tabanı
- Test ve Hata Yönetimi
- Rapor

***NOT-1:** Kopya çektiği/intihal yaptığı tespit edilen projeler **0 (sıfır)** olarak notlandırılacaktır.

***NOT-2:** GitHub'a düzenli ve ekip üyesi sayısına göre orantılı commit yapmayanlar **0 (sıfır)** olarak notlandırılacaktır.

***NOT-3:** OOP, TDD vb. prensiplere uygun geliştirme yapmayan öğrenciler/projeler **0 (sıfır)** olarak notlandırılacaktır.

***NOT-4:** Belirtilen tarihe kadar grup oluşturmayan, ekip arkadaşı olmasa dahi listeye ismini eklemeyen, proje dosyalarının gönderimini sağlamayan öğrenciler projeler **0 (sıfır)** olarak notlandırılacaktır.

***NOT-5:** Her grup yalnızca tek ve ortak bir proje geliştirip sunacaktır. Ayrı ayrı sunum alınmayacak olup ekibin tamamı **0 (sıfır)** olarak notlandırılacaktır.

7. Tarihler

Proje gruplarının oluşturulması için son tarih: 26 Şubat 2026 Cuma Saat 17.00. Link için tıklayınız.

Proje Son Teslim Tarihi: 5 Nisan 2026 Pazar Saat 23.59

Proje Sunum Tarihleri: 6 Nisan 2026 – 17 Nisan 2026 tarihleri arasında, son teslim tarihinden sonra sunum listeleri ilan edilecektir.

Ekler

https://www.markdownguide.org/
https://github.com/mermaid-js/mermaid
https://github.blog/developer-skills/github/include-diagrams-markdown-files-mermaid/
https://github.com/adam-p/markdown-here/wiki/markdown-cheatsheet
https://www.geeksforgeeks.org/software-engineering/test-driven-development-tdd/
https://microservices.io/
https://www.geeksforgeeks.org/system-design/microservices/
https://docs.docker.com/reference/cli/docker/compose/up/
https://www.geeksforgeeks.org/devops/what-is-docker-compose-up/
https://restfulapi.net/richardson-maturity-model/
https://www.geeksforgeeks.org/node-js/richardson-maturity-model-restful-api/